

~ -- -zsh

... nano quotes_parser.c

+

UW PICO 5.09File: quotes_parser.cModified

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define BUF_SIZE 512

// Token Structure[cite: 10]
typedef struct {
 char *value; // Token string value[cite: 10]
 int is_quoted; // Flag: 1 if inside quotes, 0 otherwise[cite: 10]
} Token;

// Helper to expand environment variables like \$USER or \$HOME[cite: 10]
char* expand_variable(const char *var_name) {
 char *val = getenv(var_name);
 return val ? val : "";
}

// Tokenizer handling Single and Double Quotes[cite: 10]
int tokenize_quotes(const char *input, Token tokens[]) {
 int token_count = 0;
 int len = strlen(input);
 int i = 0;

 while (i < len) {
 // Skip leading whitespace outside quotes[cite: 10]
 while (i < len && isspace((unsigned char)input[i])) i++;
 if (i >= len) break;

 char token_buf[BUF_SIZE];
 int buf_idx = 0;
 int in_single_quote = 0;
 int in_double_quote = 0;
 int is_quoted_token = 0;

 while (i < len) {
 char ch = input[i];

 // Handle Single Quote[cite: 11]
 if (ch == '\'' && !in_double_quote) {
 in_single_quote = !in_single_quote;
 is_quoted_token = 1;
 i++;
 continue;
 }

 token_buf[buf_idx] = ch;
 buf_idx++;
 i++;
 }

 Token t;
 t.value = token_buf;
 t.is_quoted = is_quoted_token;
 tokens[token_count] = t;
 token_count++;
 if (token_count == MAX_TOKENS) break;
 }
 return token_count;
}

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

Cur Pos
To Spell

~ -- -zsh

... nano quotes_parser.c

+

UW PICO 5.09File: quotes_parser.cModified

int i = 0;

while (i < len) {
 // Skip leading whitespace outside quotes[cite: 10]
 while (i < len && isspace((unsigned char)input[i])) i++;
 if (i >= len) break;

 char token_buf[BUF_SIZE];
 int buf_idx = 0;
 int in_single_quote = 0;
 int in_double_quote = 0;
 int is_quoted_token = 0;

 while (i < len) {
 char ch = input[i];

 // Handle Single Quote[cite: 11]
 if (ch == '\'' && !in_double_quote) {
 in_single_quote = !in_single_quote;
 is_quoted_token = 1;
 i++;
 continue;
 }

 // Handle Double Quote[cite: 11]
 if (ch == '"' && !in_single_quote) {
 in_double_quote = !in_double_quote;
 is_quoted_token = 1;
 i++;
 continue;
 }

 token_buf[buf_idx] = ch;
 buf_idx++;
 i++;
 }

 // Stop at unquoted whitespace[cite: 11]
 if (isspace((unsigned char)ch) && !in_single_quote && !in_double_quote) {
 break;
 }

 // Variable Expansion in Double Quotes or Unquoted strings[cite: 11]
 if (ch == '\$' && !in_single_quote) {
 i++;
 char var_name[64];
 int var_idx = 0;

 while (i < len && (isalnum((unsigned char)input[i]) || input[i] == '_')) {
 var_name[var_idx++] = input[i++];
 }
 var_name[var_idx] = '\0';

 char *expanded_val = expand_variable(var_name);
 if (expanded_val) {
 token_buf[buf_idx] = expanded_val;
 buf_idx++;
 i++;
 }
 }

 Token t;
 t.value = token_buf;
 t.is_quoted = is_quoted_token;
 tokens[token_count] = t;
 token_count++;
 if (token_count == MAX_TOKENS) break;
}

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

Cur Pos
To Spell

~ -- -zsh

...

~ -- nano quotes_parser.c

+

UW PICO 5.09File: quotes_parser.cModified

```
// Handle Double Quote[cite: 11]
if (ch == '"' && !in_single_quote) {
    in_double_quote = !in_double_quote;
    is_quoted_token = 1;
    i++;
    continue;
}

// Stop at unquoted whitespace[cite: 11]
if (isspace((unsigned char)ch) && !in_single_quote && !in_double_quote) {
    break;
}

// Variable Expansion in Double Quotes or Unquoted strings[cite: 11]
if (ch == '$' && !in_single_quote) {
    i++;
    char var_name[64];
    int var_idx = 0;

    while (i < len && (isalnum((unsigned char)input[i]) || input[i] == '_')) {
        var_name[var_idx++] = input[i++];
    }
    var_name[var_idx] = '\0';

    if (var_idx > 0) {
        char *expanded = expand_variable(var_name);
        while (*expanded && buf_idx < BUF_SIZE - 1) {
            token_buf[buf_idx++] = *expanded++;
        }
    }
    else {
        token_buf[buf_idx++] = '$'; // Literal '$' if no variable name follows[cite: 12]
    }
    continue;
}

// Append character literal[cite: 12]
token_buf[buf_idx++] = ch;
i++;
}

// Error Detection: Unclosed Quotes[cite: 12]
if (in_single_quote || in_double_quote) {
    printf("[Syntax Error] Unclosed quote detected.\n");
    return -1;
}

token_buf[buf_idx] = '\0';
tokens[token_count].value = strdup(token_buf);
```

^G Get Help

^X Exit

^O WriteOut

^J Justify

^R Read File

^W Where is

^Y Prev Pg

^V Next Pg

^K Cut Text

^U UnCut Text

^C Cur Pos

^T To Spell

~ -- -zsh

...

~ -- nano quotes_parser.c

+

UW PICO 5.09File: quotes_parser.cModified

```
        if (var_idx > 0) {
            char *expanded = expand_variable(var_name);
            while (*expanded && buf_idx < BUF_SIZE - 1) {
                token_buf[buf_idx++] = *expanded++;
            }
        }
        else {
            token_buf[buf_idx++] = '$'; // Literal '$' if no variable name follows[cite: 12]
        }
        continue;
    }

    // Append character literal[cite: 12]
    token_buf[buf_idx++] = ch;
    i++;
}

// Error Detection: Unclosed Quotes[cite: 12]
if (in_single_quote || in_double_quote) {
    printf("[Syntax Error] Unclosed quote detected.\n");
    return -1;
}

token_buf[buf_idx] = '\0';
tokens[token_count].value = strdup(token_buf);
tokens[token_count].is_quoted = is_quoted_token;
token_count++;
}

return token_count;
}

// Memory Cleanup[cite: 12]
void free_tokens(Token tokens[], int count) {
    for (int i = 0; i < count; i++) {
        free(tokens[i].value);
    }
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nQuote Parser > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline[cite: 12]
        if (strcmp(input_buf, "exit") == 0) break;
```

^G Get Help

^X Exit

^O WriteOut

^J Justify

^R Read File

^W Where is

^Y Prev Pg

^V Next Pg

^K Cut Text

^U UnCut Text

^C Cur Pos

^T To Spell

```
~ -- -zsh ... ~ -- nano quotes_parser.c +
LW PICO 5.09 File: quotes_parser.c Modified

    tokens[token_count].is_quoted = is_quoted_token;
    token_count++;
}

return token_count;
}

// Memory Cleanup[cite: 12]
void free_tokens(Token tokens[], int count) {
    for (int i = 0; i < count; i++) {
        free(tokens[i].value);
    }
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nQuote Parser > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline[cite: 12]
        if (strcmp(input_buf, "exit") == 0) break;

        Token tokens[MAX_TOKENS];
        int token_count = tokenize_quotes(input_buf, tokens);

        if (token_count >= 0) {
            printf("---- Token Output ----\n");
            for (int i = 0; i < token_count; i++) {
                printf("Token [%d]: \"%s\" (Quoted Mode: %s)\n",
                    i, tokens[i].value, tokens[i].is_quoted ? "YES" : "NO");
            }
            free_tokens(tokens, token_count);
        }

        return 0;
    }
}

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell
```

```
~ -- -zsh ... ~ -- -zsh +

medikondarakesh@Medikondas-MacBook-Air ~ % nano quotes_parser.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall quotes_parser.c -o quotes_parser
medikondarakesh@Medikondas-MacBook-Air ~ % ./quotes_parser

Quote Parser > echo '$USER is logged in'
--- Token Output ---
Token [0]: "echo" (Quoted Mode: NO)
Token [1]: "$USER is logged in" (Quoted Mode: YES)

Quote Parser > echo "USER is logged in"
--- Token Output ---
Token [0]: "echo" (Quoted Mode: NO)
Token [1]: "medikondarakesh is logged in" (Quoted Mode: YES)

Quote Parser > echo "Hello World" 'Single Quotes: $HOME'
--- Token Output ---
Token [0]: "echo" (Quoted Mode: NO)
Token [1]: "Hello World" (Quoted Mode: YES)
Token [2]: "Single Quotes: $HOME" (Quoted Mode: YES)

Quote Parser > echo "Unclosed string
[Syntax Error] Unclosed quote detected.

Quote Parser > exit
medikondarakesh@Medikondas-MacBook-Air ~ %
```

